World Models and World Action Models for Embodied AI

Training Objectives, Generative Inference, and Robot Deployment

Foundation of Embodied AI ·Lecture Notes

2026-10-19

Contents

1. Why should a robot predict the world? . . . 2
2. Four architectural paradigms . . . 2
3. What does WAM training data look like? . . . 3
4. Training a WAM end to end . . . 4
5. Three distinct inference paths . . . 5
6. Real-time systems: measure the full control loop . . . 5
7. Contact-rich manipulation and failure recovery . . . 6
8. How to assess a WAM contribution . . . 6
9. Integration with the existing course . . . 7
10. References and implementation entry points . . . 7

Course home ·简体中文 ·PDF

Item	Details
Topic	World Action Models
Revision	2026-10-19
License	CC BY 4.0 ·Xiong Zeng and contributors

1. Why should a robot predict the world?

Scope. This chapter follows the course material on imitation learning, flow matching, and GR00T/SONIC. It focuses on how predictive world representations connect to executable robot actions, how World Action Models (WAMs) are trained, and what makes their inference viable on physical robots. It does **not** repeat the general mathematics of conditional flow matching or PPO.

Reading path. Definitions and taxonomy → training data → prediction/action objectives → inference variants → systems latency → contact and recovery → research checklist.

Robotic actions affect objects, contacts, and future observations. In manipulation, the same planned motion may succeed or fail depending on grasp pose, friction, occlusion, and disturbance. A conventional vision-language-action (VLA) policy can model

$$\pi_\theta(A_t \mid h_t, \ell), \qquad h_t = (o_{t-k:t}, q_{t-k:t}), \qquad A_t = (a_t, \dots, a_{t+H-1}).$$

Here o is sensory input, q is proprioception, and ℓ is the task description. Importantly, **a VLA is not necessarily ignorant of dynamics**: behavior data and learned representations may already contain implicit physical regularities. Explicit world prediction supplies an additional temporal training signal, a counterfactual interface, or a planning mechanism.

A world model (WM) commonly predicts action-conditioned observations or latent states:

$$p_\phi(o_{t+1:t+H} \mid h_t, A_t), \qquad p_\phi(z_{t+1} \mid z_t, a_t).$$

A WAM couples predicted futures with **executable robot behavior**. This coupling can occur through imagine-then-execute, predictive features, joint video-action generation, or auxiliary prediction during training. **It does not imply that future video must be generated at test time.**

1.1 Three neighboring model classes

Model	Typical output	Training supervision	Role
VLA / action policy	Action chunks	Actions, task feedback	Select actions
World model	Future states or observations	Temporal transitions	Predict, score, plan
World action model	Future-linked executable actions	Coupled prediction and action supervision	Train and/or execute a policy

High-quality video prediction does not establish correct contact physics, action controllability, or reliable robot control. A generic video generator is therefore not automatically a deployable WAM.

2. Four architectural paradigms

2.1 Imagine then execute

First generate a desired or anticipated future $\hat{O}^+$, then condition an action policy on it:

$$\hat{O}^+ \sim p_\phi(O^+ \mid h_t, \ell), \qquad A_t \sim \pi_\theta(A \mid h_t, \ell, \hat{O}^+).$$

The predicted frames can convey intended object motion or task progress. The downside is costly video generation and potentially inconsistent or unreachable imagined states. Some actual implementations condition imagination on candidate actions; the expression above illustrates only one possible factorization.

2.2 Video-feature-conditioned action prediction

Use intermediate predictive features directly:

$$f_t = f_\phi(h_t, \ell), \qquad A_t \sim \pi_\theta(A \mid f_t, q_t).$$

This can avoid full pixel-space video decoding. However, whether the representation is *controllable* and *contact aware* must be established through interactive evaluation, not only generative video benchmarks.

2.3 Joint video-action modeling

A joint model represents

$$p_{\theta,\phi}(O^+, A \mid h_t, \ell).$$

Possible realizations include shared latent backbones, separate heads, alternating token prediction, and coupled denoising. **Joint modeling does not prescribe a unique video/action generation order**. Shared world representations may improve action prediction, but data alignment and inference cost remain central.

2.4 Auxiliary future prediction during training

Use prediction as an auxiliary loss and skip explicit video prediction during execution:

$$\min_{\theta,\phi} \ \mathcal{L}_{\text{action}} + \lambda \mathcal{L}_{\text{future}}, \qquad A_t \leftarrow \pi_\theta(h_t, \ell) \quad \text{at test time.}$$

Fast-WAM illustrates why the advantages of predictive training should be distinguished from the cost of test-time imagination. Its findings are empirical rather than a universal guarantee for every embodiment and task.

3. What does WAM training data look like?

A robot interaction dataset may contain

$$\mathcal{D}_R = \{(o_t, q_t, a_t, o_{t+1}, r_t, d_t, \ell, \Delta t)\}_{t=0}^{T-1}.$$

The reward r_t is optional for supervised pretraining and primarily useful for RL, planning, or ranking. d_t marks episode termination and Δt records physical time. The action field must identify units, reference frames, control mode, and execution timestamp. Distinguish commanded actions from actions actually applied after saturation, filtering, and communication delay.

3.1 Sources and supervision

Source	Useful learning signal	Major limitation
Human or online video	Visual dynamics and interaction priors	Missing robot action labels
Robot demonstrations	Paired sensor/action trajectories	Limited embodiments and tasks
Simulator rollouts	State, action, contact, reward	Reality gap in friction and appearance
Online robot episodes	Failures and recovery states	Safety, time, and annotation costs

For video-only data, mask unavailable action losses. Inferred actions from inverse dynamics or retargeting are **pseudo-labels**, not ground-truth robot commands; their uncertainty and embodiment should be recorded. Misaligned camera timestamps and action execution timestamps can create systematic errors in the learned causal relationship.

3.2 Representation and synchronization

Inputs may include RGB/depth images, compressed video latents, 3D object poses, joint states, tactile readings, and control history. Outputs may be joint increments, end-effector SE(3) targets, motion tokens, or lower-level references. An action type is part of the model contract. Never silently replace position targets with velocities or change the coordinate frame at deployment. Multi-rate inputs require synchronization rather than naive index matching.

4. Training a WAM end to end

4.1 Stage A: pretrain predictive representations

For a latent predictive model, let $z_t = E_\eta(o_t)$ and predict

$$\hat{z}_{t+1} = F_\phi(z_{\le t}, a_{\le t}), \qquad \mathcal{L}_z = \sum_t \|\hat{z}_{t+1} - \mathrm{sg}(E_\eta(o_{t+1}))\|_2^2.$$

This is an **illustrative** latent regression objective. Alternatives include autoregressive cross-entropy, contrastive or JEPA-style prediction, diffusion noise prediction, and flow-matching velocity regression. Stop-gradient is model dependent. Video-only pretraining can learn temporal structure, but it cannot by itself identify arbitrary robot action interventions.

4.2 Stage B: train the action expert

Let A^* denote an expert action chunk and ε a Gaussian sample. A simple conditional flow-matching construction is

$$A_\tau = (1-\tau)\varepsilon + \tau A^*,$$

$$\mathcal{L}_{\text{act}} = \mathbb{E}_{A^*,\varepsilon,\tau}\left[\|v_\theta(A_\tau, \tau, c_t) - (A^* - \varepsilon)\|_2^2\right],$$

where c_t contains predictive features, robot state, and instruction. The generation-time variable τ is **not physical robot time**, and the predicted flow velocity is **not joint velocity**. See the separate Flow Matching chapter for a derivation of probability paths and ODE sampling.

4.3 Stage C: couple the prediction and action objectives

A schematic—not universal—joint loss is

$$\mathcal{L} = \lambda_a \mathcal{L}_{\text{act}} + \lambda_v \mathcal{L}_{\text{video}} + \lambda_z \mathcal{L}_{\text{latent}} + \lambda_c \mathcal{L}_{\text{consistency}}.$$

A video objective may be diffusion noise prediction or flow velocity regression. Consistency could relate predicted futures to actions through a known transition model, a learned differentiable predictor, or a contrastive objective. **The consistency term is not computable without defining its mechanism.** Missing supervision should be masked; the video objective should not overwhelm task-relevant action learning. Common failure modes include stale predictive features, action normalization errors across embodiments, and compounding long-horizon prediction errors.

4.4 Stage D: task adaptation and optional RL

Supervised adaptation fits action outputs to the target robot and task. RL may update an action expert, a residual branch, or a supervisory policy using interaction feedback. World-model planning can rank candidate actions by predicted futures and rewards; however, a policy may exploit inaccuracies in a learned predictor. Evaluate predictive fidelity and real-environment improvement separately.

4.5 Specify the training contract

Lock down sensor timestamps, action semantics, observation history, prediction horizon, camera extrinsics, normalization, train/test splits, and embodiment-specific mappings. Include held-out objects, initial poses, disturbances, and recovery examples. Video fidelity metrics such as FVD or PSNR are insufficient for evaluating contact-rich control policies.

5. Three distinct inference paths

5.1 Explicit future imagination

Observe h_t, sample a future video or latent rollout $\hat{O}^+$, use it to generate an action chunk, execute only the first $K \leq H$ actions, and observe again. This may provide interpretable prospective behavior but adds video sampling latency and risks unrealistic imagined contact outcomes.

5.2 Joint video-action generation

Generate $(\hat{O}^+, \hat{A})$ with coupled visual/action tokens or denoising states. Specify the factorization, causal mask, and generation schedule. **Joint generation is not automatically model predictive control (MPC)**: MPC additionally requires the evaluation or optimization of candidates in a receding-horizon decision loop.

5.3 Action-only inference

Use a representation learned with future prediction but omit expensive future decoding:

$$\hat{A}_t = G_\theta(h_t, q_t, \ell).$$

Assess the number of backbone calls, ODE/denoising evaluations, and dependence on predictive intermediate features. Fast-WAM reports approximately 190 ms latency in its evaluated configuration; this is **not a generic WAM latency target**. Paper.

6. Real-time systems: measure the full control loop

End-to-end latency should be decomposed as

$$T_{\text{e2e}} = T_{\text{sense}} + T_{\text{encode}} + T_{\text{world}} + T_{\text{action}} + T_{\text{network}} + T_{\text{dispatch}}.$$

The quantity that matters for a robot is the delay from sensing to **command actuation**, not just GPU throughput. During asynchronous inference, a previously issued action chunk may be executing while the next chunk is computed. The controller must maintain physically continuous commands or provide a safe fallback.

6.1 What can be accelerated?

Method	Cost reduced	Key caveat
Skip explicit video generation	Video sampling	Less explicit future checking
Reduce sampling steps	Flow/diffusion evaluations	Possible quality degradation
KV cache for causal backbones	Redundant prefix attention	Invalidation under new inputs
FlashAttention	Attention memory traffic	Not equivalent to whole-model speedup
Reuse denoising/replanning features	Redundant backbone evaluations	Staleness and approximation error
Distillation or quantization	Compute and memory	Tail latency and contact precision

Ordinary autoregressive **KV caching** should not be conflated with cross-denoising-step caches in bidirectional diffusion transformers. Similarly, FlashAttention optimizes attention kernels but does not remove the need for multiple video or

action sampling steps. The separate efficient-training-and-inference lecture should handle kernel details, profiling, quantization, and cache validity more deeply.

6.2 Respect the robot clock

Let the low-level control period be Δt_c, the prediction horizon H, the executed chunk length K, and model latency d control periods. New predictions must be synchronized with already committed actions. The imitation-learning chapter covers real-time chunking (RTC) and prefix inpainting; WAMs create additional timing constraints when future video prediction lies on the critical path. Never assume that smooth commanded trajectories imply safe contact forces.

7. Contact-rich manipulation and failure recovery

RGB images generally do not uniquely determine normal force, friction margin, or incipient slip. This partial observability matters for dexterous grasping, tool use, and insertion. One possible task-focused latent state is

$$z_t = (T_{\text{obj}}, T_{\text{hand}}, q_t, \hat{c}_t, \Sigma_t),$$

where $\hat{c}_t$ is an estimated contact mode and Σ_t captures uncertainty. Contact supervision can be collected from a simulator, tactile readings, or event labeling, but is not magically available for ordinary video data. A good world-action evaluation should cover free-space motion, contact onset, stable manipulation, slip, object drops, and recovery.

7.1 Running example: grasp and rotate a screwdriver

The high-level goal is to locate the tool, establish a feasible grasp, move to a designated pose, and rotate. The predictive branch estimates object motion and failure risks under candidate approaches. The action branch proposes end-effector and hand commands; a faster feedback controller maintains joint and grasp stability. If visual prediction says "the screwdriver is grasped" while motor-current or position residuals indicate slip, the system should pause, reobserve, or execute a recovery policy rather than blindly committing to the remainder of an action chunk.

7.2 Minimum real-robot deployment checks

1. Fix action type, frame, frequency, and safety limits across simulation and hardware.
2. Timestamp camera exposure, encoding, inference, network transport, and command actuation.
3. Report $p50/p95/p99$ end-to-end latency and define inference-timeout fallback behavior.
4. Distinguish good predicted futures, successful actions, and actual task completion.
5. Treat world-model uncertainty as a risk signal, not a substitute for reliable contact sensing or a hardware safety layer.

8. How to assess a WAM contribution

Dimension	Useful metrics	Common confound
Prediction	Short/long horizon calibration, motion agreement	Visual quality mistaken for physics
Policy	Success, task and embodiment generalization	In-distribution only evaluation
Robustness	Occlusion, perturbation, recovery	Open-loop replay
Systems	End-to-end p95 latency, control continuity	Forward-pass latency only
Data efficiency	New-robot demonstrations and total data costs	Ignoring pretrained data
Safety	Collisions, forces, timeout recovery	Assuming video makes control safe

At minimum, ablate video pretraining, future-prediction auxiliary training, explicit future prediction at inference, and closed-loop replanning. These interventions test distinct mechanisms. A single "WAM versus VLA" benchmark does not establish which mechanism causes an improvement.

9. Integration with the existing course

The Flow Matching lecture provides the probability-path and velocity-field derivations. Imitation Learning covers BC, DAgger, action chunking, and RTC. RL Post-training covers which pretrained components to freeze or update. GR00T/SONIC covers the System 1 to System 0 action interface. **This lecture owns the predictive-world-to-action coupling**, and the Robot Agent lecture covers task-level orchestration, tool use, verification, and recovery policies.

Recommended sequence: predictive problem definition → four paradigms → dataset contract → objectives → inference path selection → latency and safety budgeting → contact/perturbation evaluation. Before implementing a WAM, decide whether the application needs interpretable test-time imagination or primarily benefits from predictive representation learning.

10. References and implementation entry points

1. Zhang, Zeng, Zhang. From World Models to World Action Models: A Concise Tutorial for Robotics, 2026. Definitions and architectural taxonomy.
2. Ye et al. World Action Models are Zero-shot Policies (DreamZero), 2026. Joint world/action modeling with video diffusion.
3. Yuan et al. Fast-WAM: Do World Action Models Need Test-time Future Imagination?, 2026. Separating training-time future prediction from test-time imagination.
4. NVIDIA. Cosmos, infrastructure for world/video generation, not a single drop-in robot policy.
5. Physical Intelligence. openpi, implementation reference for flow-based robot action experts.
6. Chi et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion, 2023.
7. Lipman et al. Flow Matching for Generative Modeling, 2023.

Note. Several equations are schematic teaching abstractions, not verbatim objectives from individual papers. Hardware latency depends on implementation, robot interfaces, and evaluation conditions. This lecture does not claim to have trained or validated a WAM on physical robots.